Screenshot checklist

LifeLog CI/CD project. Everything you need to capture, in the order you will naturally meet it.

How to capture and name them

1. On the Mac press **Cmd+Shift+4** (drag a rectangle) or **Cmd+Shift+5**. Capture only the area that matters, but keep it readable.
2. Save each file as **PNG** with the **exact file name** in the table, into `~/lifelog-cicd/docs/screenshots/`.
3. Either rebuild the PDFs yourself with `~/mpft/.venv/bin/python scripts/make_docs.py`, or send the files to me and I will rebuild them.

> **Never show secrets.** The Docker Hub token must not appear in any screenshot. Do not open the `dockerhub-creds` credential. Your EC2 public IP is fine to show.

You need **20 required** screenshots and 2 optional ones. The 12 app screens (light and dark theme, capture bar, log, insights, trackers, goals, mobile) are already captured and included.

AWS

#	File name (.png)	How to capture	Must be visible	Need
1	`aws-01-ec2-instance`	AWS Console, EC2, Instances, select lifelog-jenkins.	Name, state Running, 2/2 checks passed, type m7i-flex.large, public IPv4, region.	required
2	`aws-02-security-group`	EC2, your instance, Security tab, Inbound rules.	Ports 22, 8080 and 30080, each with source My IP.	required
3	`aws-03-tool-versions`	SSH terminal: java -version 2>&1 / head -1; docker --version; kubectl version --client / head -1; minikube version / head -1; systemctl is-active jenkins	All five lines in one screenshot: Java 21, Docker, kubectl, Minikube, and 'active'.	required

Jenkins

#	File name (.png)	How to capture	Must be visible	Need
4	`jenkins-01-dashboard`	Open http://:8080 after login.	The lifelog-pipeline job with a green status icon and the left menu.	required
5	`jenkins-02-plugins`	Manage Jenkins, Plugins, Installed plugins, search 'pipeline'.	Docker Pipeline and Pipeline: Stage View in the list.	required
6	`jenkins-03-credentials`	Manage Jenkins, Credentials, Global.	The row dockerhub-creds. Never open it and never show the token.	required
7	`jenkins-04-job-config`	Job, Configure, scroll to the Pipeline section.	Pipeline script from SCM, Git, the GitHub repo URL, branch */main, Script Path Jenkinsfile.	required

#	File name (.png)	How to capture	Must be visible	Need
8	`jenkins-05-stage-view`	Open the job page and look at Stage View.	Every stage green: Checkout, Build & Test, Docker Build, Push Image, Deploy to Minikube, Verify.	required
9	`jenkins-06-console-success`	Open a build, Console Output, scroll to the bottom.	The Verify stage JSON {"status":"ok",...} and 'Finished: SUCCESS'.	required
10	`jenkins-07-console-tests`	Console Output, find the Build & Test stage.	The pytest summary line, for example '65 passed'.	required

Docker

#	File name (.png)	How to capture	Must be visible	Need
11	`docker-01-local-images`	On your Mac: docker compose up -d --build, then docker images / grep lifelog.	lifelog-backend and lifelog-frontend with sizes.	required

Docker Hub

#	File name (.png)	How to capture	Must be visible	Need
12	`dockerhub-01-repos`	hub.docker.com, your profile, Repositories.	lifelog-backend and lifelog-frontend, both Public.	required
13	`dockerhub-02-tags`	Open lifelog-backend, Tags tab.	Tags named with build numbers (1, 2, 3...) plus latest, with push times.	required

Kubernetes

#	File name (.png)	How to capture	Must be visible	Need
14	`k8s-01-minikube-status`	On the instance: sudo -u jenkins -H minikube status; sudo -u jenkins -H kubectl get nodes	host/kubelet/apiserver Running and a node with status Ready.	required
15	`k8s-02-get-all`	On the instance: sudo -u jenkins -H kubectl get pods,deploy,svc	2 frontend pods, 1 backend pod, all Running 1/1, and service lifelog as NodePort 80:30080/TCP.	required
16	`k8s-03-describe-probes`	sudo -u jenkins -H kubectl describe deploy lifelog-backend / grep -E 'Replicas/Image/Liveness/Readiness/Limits/Requests' -A1	The image with a build-number tag, the probes on /health, and the limits.	required

GitHub

#	File name (.png)	How to capture	Must be visible	Need
17	`github-01-repo-branches`	GitHub repo page, click the branch dropdown or the Branches link.	main and v2-goals.	required
18	`github-02-commits`	GitHub repo, Commits, after the second-run merge.	The 'Merge v2-goals: add Goals module' commit.	optional

Live app on AWS

#	File name (.png)	How to capture	Must be visible	Need
19	`app-aws-01-before`	Open http://:30080 and scroll to the footer.	The page and the footer: version, build number, commit, frontend pod, backend pod. No Goals link.	required
20	`k8s-04-rolling-update`	During the second run, on the instance: sudo -u jenkins -H kubectl get pods -w	Old pods Terminating and new pods ContainerCreating or Running.	optional
21	`jenkins-08-two-runs`	Job page after the second run.	Two green rows with different build numbers and the second one listing the merge commit.	required
22	`app-aws-02-after`	Reload http://:30080/goals after build is green.	Goals page, Goals in the nav, and the footer with the new build number and new pod names.	required

Order of work

1. Take the AWS, Jenkins, Docker Hub, GitHub and Kubernetes shots while the first successful run is on screen.
2. Take **app-aws-01-before** and **jenkins-05-stage-view** *before* the second run.
3. Do the second run (merge `v2-goals`, push, Build Now) and take **k8s-04-rolling-update** while it deploys.
4. When it is green take **app-aws-02-after**, **jenkins-08-two-runs** and **github-02-commits**.

Already captured for you

File	What it shows
`app-today-light`	Today view, light theme
`app-today-dark`	Today view, dark theme
`app-capture-preview`	Capture bar with live preview
`app-capture-create`	Unknown name offers one-click create
`app-capture-ambiguous`	Ambiguous name shows suggestions
`app-log`	Log view
`app-insights`	Insights view
`app-trackers`	Trackers list
`app-tracker-detail`	Tracker detail with heatmap
`app-tracker-form`	New tracker form
`app-goals`	Goals page (v2-goals branch)
`app-mobile-today`	Mobile layout